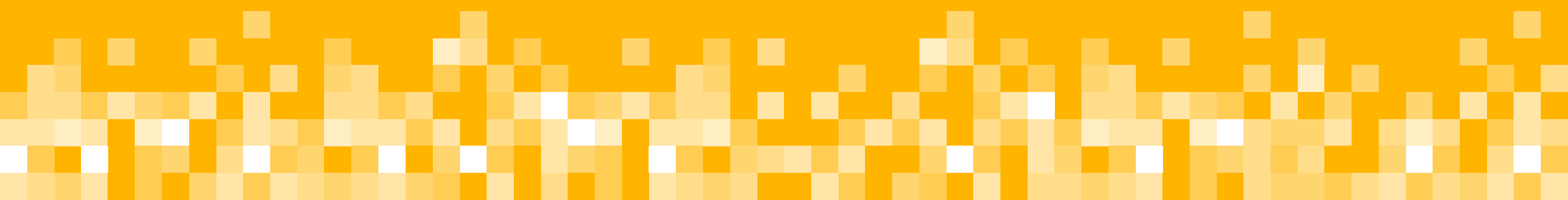


Moduł 3

Refaktoryzacja do wzorca Aggregate

Ćwiczenia praktyczne



Zadanie:

Podział obiektu

Wykonaj komendę:

```
git checkout podzial-zadanie
```

w odpowiednim projekcie:

- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-java>
- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-php>
- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-csharp>

i spójrz na zrefaktoryzowaną już kiedyś klasę `legacyfighter.newproducts.OldProduct`. Spójrz też na klasę: `legacyfighter.newproducts.OldProductService`. Zastanów się, jakie odpowiedzialności ma klasa `OldProduct`. Wypisz je i do każdej z tych odpowiedzialności zlokalizuj miejsce w kodzie tej klasy. Czy widzisz jakieś problemy? Warto dodać, że w procesie wytwarzania oprogramowania w naszej firmie (nie tylko w samym kodzie), występują następujące problemy:

- 1) Częste zmiany tej klasy powodujące konflikty przy "merge'u"
- 2) Zmiany psują niezwiązane ze sobą funkcje biznesowe (magazynier narzeka, że coś się zepsuło, po wypuszczeniu nowej funkcji dla marketingowców)

To oznacza, że klasa `OldProduct` jest zbyt duża. Zaplanuj i dokonaj odpowiednich zmian. Plan i wykonanie zmian powinno skutkować usunięciem problemów z wieloma odpowiedzialnościami. Pamiętaj o:

- 1) Zapewnieniu, że klasa po zmianach zachowuje się tak samo - jak to sprawdzić?
- 2) Zobacz się ilu bezpośrednich klientów (używających metod biznesowych) ma klasa `OldProduct`. Czy jest to tylko `OldProductService`? Załóżmy, że jest ich naprawdę dużo (tak, jak w realnym systemie). Na tyle dużo, że, podłączenie jest kluczowym i najbardziej ryzykownym krokiem refaktoryzacji - warto więc zastosować sztuczkę ze wzorcem proxy
- 3) Fakcie, że każda refaktoryzacja jest łatwiejsza jeśli zaplanujesz w głowie szereg kolejnych mniejszych przekształceń.

Przykładowe (poglądowe, nie wzorcowe) rozwiązanie znajdziesz wykonując komendę:

```
git checkout podzial-rozwiazanie
```